



Unidad 4

Clases anónimas, lambdas y adaptadores

Prof: Lic. Rosales Pablo (prosales@unpata.edu.ar)
Materia: Programación Orientada a Objetos
Año: 2026





La pre clase, en dos preguntas

- La caja tiene dos botones, "Efectivo" y "Debito". Un solo `actionPerformed()` los atiende con un `if` por `getSource()`. Funciona.
- El local empieza a cobrar con QR. ¿Qué hay que abrir para agregar el tercer botón? ¿Qué principio SOLID falla?
- El cajero pide que Enter registre en efectivo. ¿Entra en ese `if`?

Cada botón nuevo abre un método que ya funcionaba. Y el teclado ni siquiera entra.



El problema: un receptor para todo

```
public ControladorCaja(VistaCaja vista, Caja caja) {
    this.vista.getBotonEfectivo().addActionListener(this);
    this.vista.getBotonDebito().addActionListener(this);
}

public void actionPerformed(ActionEvent evento) {
    double monto = Double.parseDouble(this.vista.getTextoMonto());
    if (evento.getSource() == this.vista.getBotonDebito()) {
        this.caja.registrar(new PagoDebito(monto));
    } else {
        this.caja.registrar(new PagoEfectivo(monto));
    }
    this.vista.mostrarTotal(this.caja.total());
}
```

Es la cadena de `if` de la clase 7, adentro de un controlador. Open/Closed falla igual.

Un receptor por evento, escrito donde se registra.

Tres formas de escribir el mismo objeto: con nombre,
sin nombre, y solo el cuerpo.



Primer intento: una clase por botón

```
public class ReceptorEfectivo implements ActionListener {
    private final VistaCaja vista;          // los mismos dos atributos
    private final Caja caja;                // que el controlador
    public ReceptorEfectivo(VistaCaja vista, Caja caja) { ... }

    public void actionPerformed(ActionEvent evento) {
        // leer el monto, registrar PagoEfectivo, mostrar el total
    }
}

// ReceptorDebito.java: igual, con PagoDebito. Y en el controlador:
vista.getBotonEfectivo().addActionListener(new ReceptorEfectivo(vista, caja));
vista.getBotonDebito().addActionListener(new ReceptorDebito(vista, caja));
```

El `if` desapareció. El costo: un archivo por botón, veinte líneas por cada una que importa.



Clase anónima: declarar y crear en el mismo lugar

```
public ControladorCaja(VistaCaja vista, Caja caja) {  
    // ...  
    this.vista.getBotonEfectivo().addActionListener(new ActionListener() {  
        @Override  
        public void actionPerformed(ActionEvent evento) {  
            registrarEfectivo();           // sin this: aca this es el anonimo  
        }  
    });  
}  
  
private void registrarEfectivo() {  
    // lo que hacia ReceptorEfectivo.actionPerformed()  
}
```

`new ActionListener() { ... }` crea un objeto de una clase sin nombre. Usa lo que tiene el controlador.



Qué es una clase anónima

- Una clase declarada y creada en el mismo lugar, sin nombre. **new** **Interfaz()** { métodos } o **new ClaseBase()** { métodos }.
- Como está adentro del controlador, usa sus atributos y sus métodos, privados incluidos. No necesita constructor ni atributos propios.
- Adentro, **this** es el objeto anónimo. Lo del controlador va sin **this**.
- De las veinte líneas de **ReceptorEfectivo** quedan seis, al lado del botón que atienden.

Una clase anónima es una clase declarada y creada en el mismo lugar, y ese lugar es donde se registra.



Lambda: solo el cuerpo

```
// De las seis lineas, cinco son sintaxis que el compilador ya conoce.
this.vista.getBotonDebito().addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent evento) {
        registrarDebito();
    }
});

// Lo unico que no puede adivinar es el cuerpo:
this.vista.getBotonDebito().addActionListener(evento -> this.registrarDebito());
// Y lo mismo para el efectivo, en una linea.

//  parametros    ->  cuerpo          (una expresion, o un bloque entre llaves)
```

Una lambda es el cuerpo del único método de una interfaz. El compilador pone el resto.



Cuándo se puede usar una lambda

- Solo si la interfaz tiene **un** método abstracto. Con dos, el compilador no sabe cuál es el cuerpo.
- A esas se las llama interfaces funcionales. **ActionListener** lo es. **Notificador** de la clase 8 también: `new Caja(mensaje -> { System.out.println(mensaje); return true; })`.
- Los parámetros son los del método; el tipo no se escribe. **this** sigue siendo el controlador.
- Cada lambda es un objeto que implementa la interfaz. Swing la llama con cada click, igual que a las otras dos formas.

Un método abstracto, lambda. Más de uno, no.

El controlador, con un receptor por botón

```
public class ControladorCaja {                                // ya no implementa nada
    private final VistaCaja vista;
    private final Caja caja;

    public ControladorCaja(VistaCaja vista, Caja caja) {
        this.vista.getBotonEfectivo()
            .addActionListener(evento -> this.registrarEfectivo());
        this.vista.getBotonDebito()
            .addActionListener(evento -> this.registrarDebito());
    }

    private void registrarEfectivo() { ... } // leer, registrar, mostrar
    private void registrarDebito() { ... }  // leer, registrar, mostrar
}
```

Sin `actionPerformed()`. Un receptor por botón en una línea, y un método privado por cosa que puede pasar. El QR es una línea y un método.



Interfaces con varios métodos

- El Enter es un evento de teclado. La interfaz es **KeyListener**, con tres métodos: **keyTyped()**, **keyPressed()**, **keyReleased()**.
- Implementarla obliga a escribir los tres, aunque dos queden vacíos. Y una lambda no sirve: son tres.
- **KeyAdapter** es una clase que implementa **KeyListener** con los tres métodos vacíos. Se hereda de ella y se sobrescribe solo el que interesa.
- Lo mismo para el mouse: **MouseListener** tiene cinco, **MouseAdapter** los trae vacíos.

Con un solo método, lambda. Con varios, una clase anónima sobre el adaptador.

Enter registra en efectivo

```
// En la vista: el campo tambien es fuente de eventos, y se expone
public JTextField getCampoMonto() { return this.campoMonto; }

// En el controlador: clase anonima que hereda de KeyAdapter
this.vista.getCampoMonto().addKeyListener(new KeyAdapter() {
    @Override
    public void keyPressed(KeyEvent evento) {
        if (evento.getKeyCode() == KeyEvent.VK_ENTER) {
            registrarEfectivo(); // el mismo metodo que el boton
        }
    }
});
```

KeyEvent dice qué tecla fue. Hay una constante por tecla: **VK_ENTER**, **VK_UP**, **VK_ESCAPE**.



Una lambda que no es de un botón

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        Caja caja = new Caja(new ServicioCorreo("smtp.lapampa.com", 587));  
        VistaCaja vista = new VistaCaja();  
        ControladorCaja controlador = new ControladorCaja(vista, caja);  
        vista.mostrar();  
    });  
    // main termina aca. La ventana sigue: Swing dejo un hilo esperando.  
}  
  
// invokeLater() recibe un Runnable: un solo metodo, run(), sin parametros.  
// Lo ejecuta en el hilo de Swing, el mismo que llama a los receptores.
```

`invokeLater()` recibe qué hacer, no cuándo. Lo mismo que un botón.



Resumen

- Un **if** por fuente adentro de un receptor es la cadena de **if** de la clase 7. Open/Closed falla.
- Clase con nombre, clase anónima y lambda son el mismo objeto para Swing. Cambia cuánto se escribe y dónde queda.
- Lambda solo con interfaces de un método abstracto. **this** sigue siendo el controlador.
- Con varios métodos, clase anónima sobre el adaptador. Adentro, **this** es el anónimo.
- El controlador ya no implementa **ActionListener**: tiene un receptor por evento y un método por cosa que puede pasar.



La actividad de hoy

- El conversor de la clase pasada, con el `if` por `getSource()` en el controlador.
- **Parte 1:** qué hay que abrir para agregar los reales, y por qué el Enter no entra en ese `if`.
- **Parte 2:** el receptor de euros pasa por las tres formas: clase con nombre, anónima, lambda. Después Enter con **KeyAdapter** y doble click con **MouseAdapter**.
- **Parte 3:** los reales. Una lambda y un método en el controlador, sin modificar nada de lo que ya estaba. Y contar los `setBounds()` que hubo que correr.